Mount Kenya University

BIT4133: NATURAL LANGUAGE PROCESSING WITH DEEP LEARNING

STUDENT NAME: IAN CHEGE

ADMISSION NUMBER: BSCCS/2024/32935

COURSE: BSc COMPUTER SCIENCE

UNIT CODE: BIT4133

UNIT NAME: NATURAL LANGUAGE PROCESSING WITH DEEP LEARNING

LECTURER: MICHAEL NYORO

SEMESTER/ACADEMIC YEAR: SEMESTER 4.2/2025-2026

PROJECT AREA

AI BIBLE STUDY COMPANION: SCRIPTURE ANALYSIS USING NLP

RESEARCH AREA: INFORMATION EXTRACTION & QUESTION GENERATION

TECHNOLOGIES USED

PYTHON | NLTK | FASTAPI | REACT + VITE | OPENAI API (GPT-4O-MINI) | RECHARTS

GITHUB LINK: https://github.com/Ian-Chege/NLP-Product

TABLE OF CONTENT

WEEK 1: INTRODUCTION TO NLP AND APPLICATIONS

Introduction to NLP Concepts and Text Processing

Fig 1: Python Installation

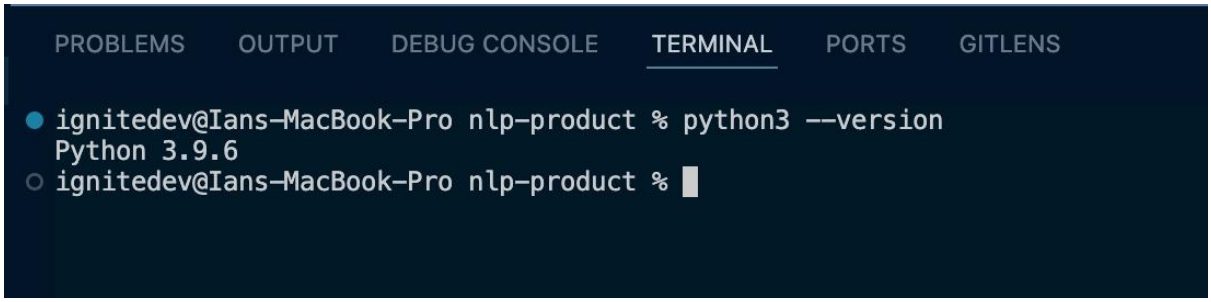


Fig 1: *Shows that Python is installed in the system.*

Fig 2: Local Development Setup

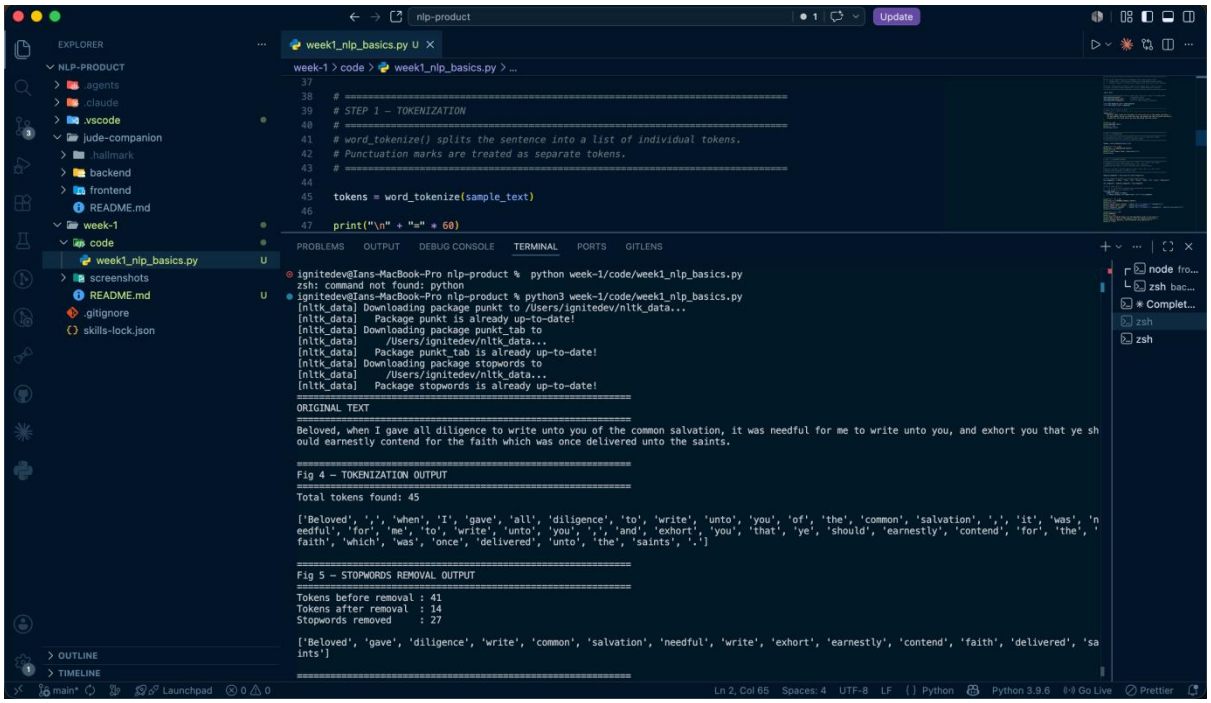


Fig 2: *Shows local development setup in VS Code.*

Fig 3: NLTK Installation

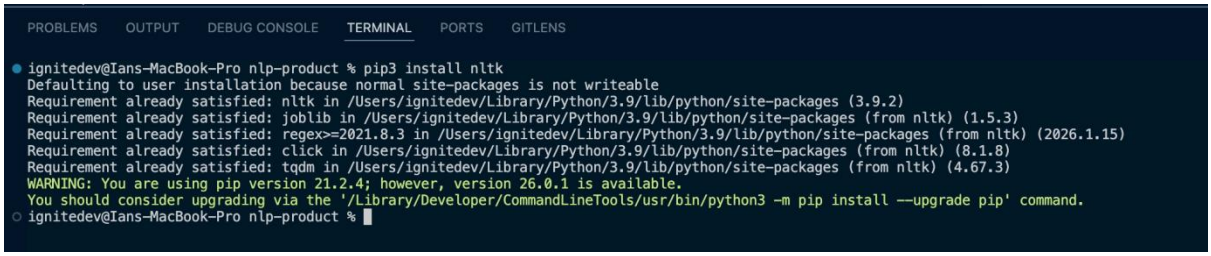


Fig 3: *Shows NLTK library installation.*

Fig 4: Tokenization Output

```
============================================================
ORIGINAL TEXT
============================================================
Beloved, when I gave all diligence to write unto you of the common salvation, it was needful for me to write unto you, and exhort you that ye sh
ould earnestly contend for the faith which was once delivered unto the saints.

============================================================
Fig 4 — TOKENIZATION OUTPUT
============================================================
Total tokens found: 45

['Beloved', ',', 'when', 'I', 'gave', 'all', 'diligence', 'to', 'write', 'unto', 'you', 'of', 'the', 'common', 'salvation', ',', 'it', 'was', 'n
eedful', 'for', 'me', 'to', 'write', 'unto', 'you', ',', 'and', 'exhort', 'you', 'that', 'ye', 'should', 'earnestly', 'contend', 'for', 'the', '
faith', 'which', 'was', 'once', 'delivered', 'unto', 'the', 'saints', '.']
```

Fig 4: *Demonstrates tokenization of a verse from Jude 1:3 using NLTK*

Fig 5: Stopwords Removal Output

```
============================================================
Fig 5 — STOPWORDS REMOVAL OUTPUT
============================================================
Tokens before removal : 41
Tokens after removal  : 14
Stopwords removed     : 27

['Beloved', 'gave', 'diligence', 'write', 'common', 'salvation', 'needful', 'write', 'exhort', 'earnestly', 'contend', 'faith', 'delivered', 'sa
ints']
```

Fig 5: *Shows the token list after filtering out common English words (the, and, was) and KJV-specific words (unto, ye, thee).*

Fig 6: NLP Application Research Evidence

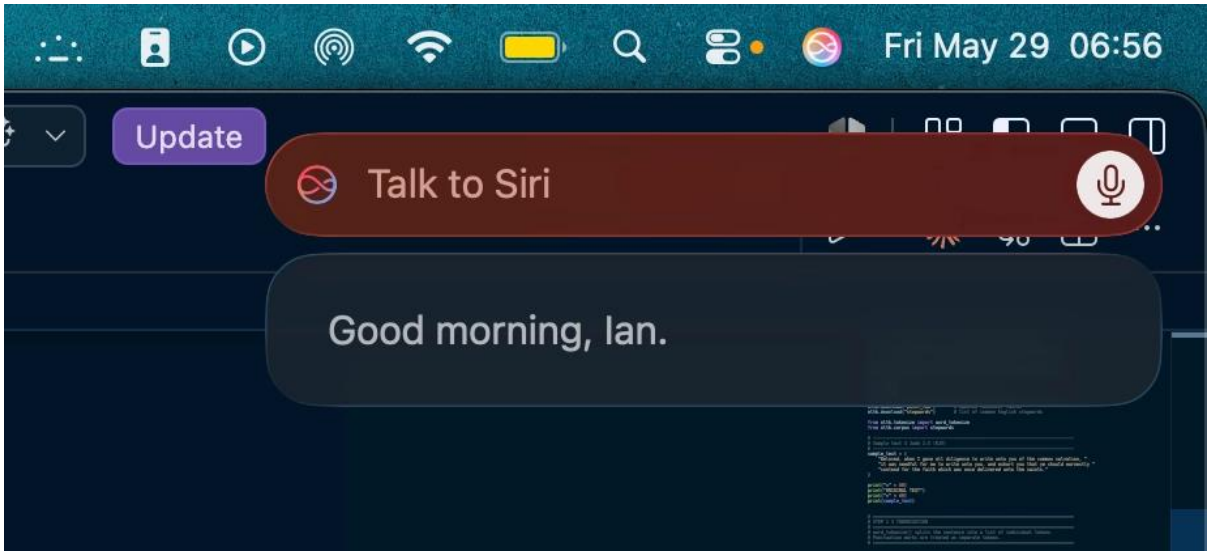


Fig 6: *Shows an interaction with Siri, an NLP application that uses speech recognition and intent detection to understand and respond to natural language in real time.*

Student Reflection

This week introduced core NLP concepts, tokenization and stopword removal using Python and the NLTK library. I applied these techniques to a verse from the Book of Jude, the same text my main project analyses. Tokenization breaks a sentence into individual words so a program can process them one at a time. Stopword removal then filters out common words like 'the' and 'and', leaving only meaningful terms. I also explored real-world NLP applications such as Siri, which uses speech recognition and intent detection to understand and respond to natural language as shown in Fig 6 above.

WEEK 2: N-GRAM MODELS AND POS TAGGING

Language Modeling and Sentence Structure Analysis

Fig 1: Bigram or Trigram Output

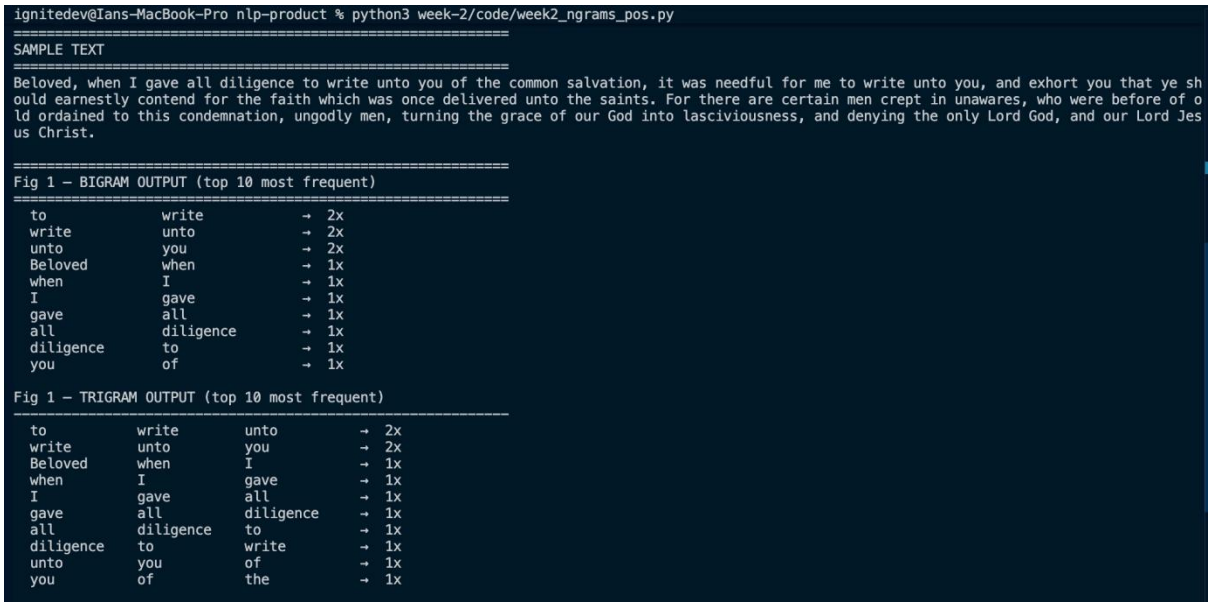
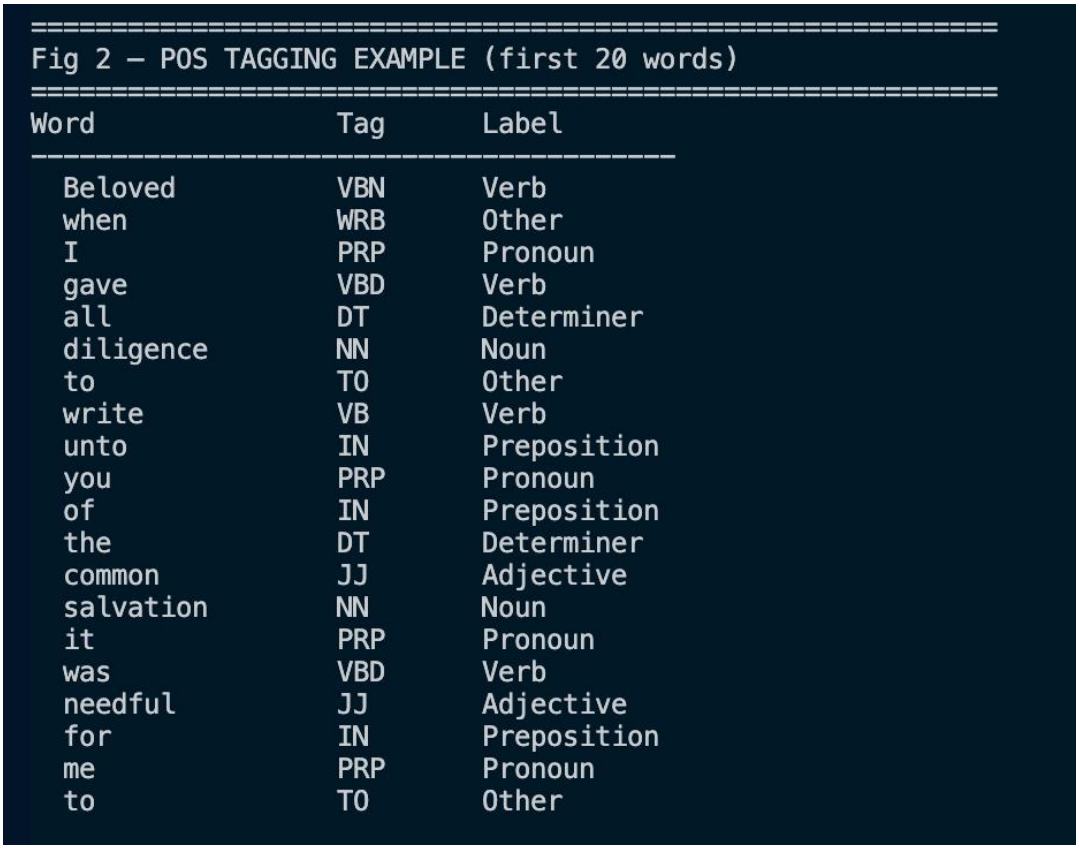


Fig 1: *Shows the most frequent two-word (bigram) and three-word (trigram) sequences found in Jude 1:3-4.*

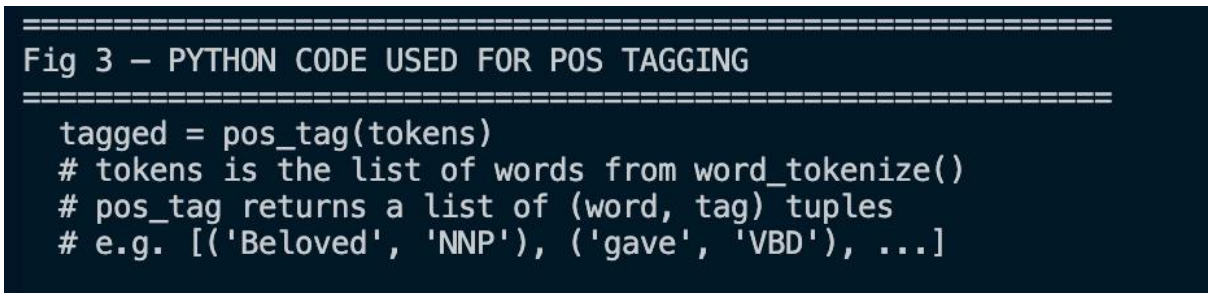
Fig 2: POS Tagging Example



```
============================================================
Fig 2 — POS TAGGING EXAMPLE (first 20 words)
============================================================
Word                Tag       Label
------------------------------------
  Beloved           VBN       Verb
  when              WRB       Other
  I                 PRP       Pronoun
  gave              VBD       Verb
  all               DT        Determiner
  diligence         NN        Noun
  to                TO        Other
  write             VB        Verb
  unto              IN        Preposition
  you               PRP       Pronoun
  of                IN        Preposition
  the               DT        Determiner
  common            JJ        Adjective
  salvation         NN        Noun
  it                PRP       Pronoun
  was               VBD       Verb
  needful           JJ        Adjective
  for               IN        Preposition
  me                PRP       Pronoun
  to                TO        Other
```

Fig 2: *Shows the first 20 words from the verse, each labelled with its grammatical role - Noun, Verb, Adjective, etc.*

Fig 3: Python Code for POS Tagging



```
============================================================
Fig 3 — PYTHON CODE USED FOR POS TAGGING
============================================================
  tagged = pos_tag(tokens)
  # tokens is the list of words from word_tokenize()
  # pos_tag returns a list of (word, tag) tuples
  # e.g. [('Beloved', 'NNP'), ('gave', 'VBD'), ...]
```

Fig 3: *Shows the core line of code used: pos_tag(tokens) which returns a list of (word, tag) tuples, e.g. [('Beloved', 'NNP'), ('gave', 'VBD')].*

Fig 4: Language Prediction Example

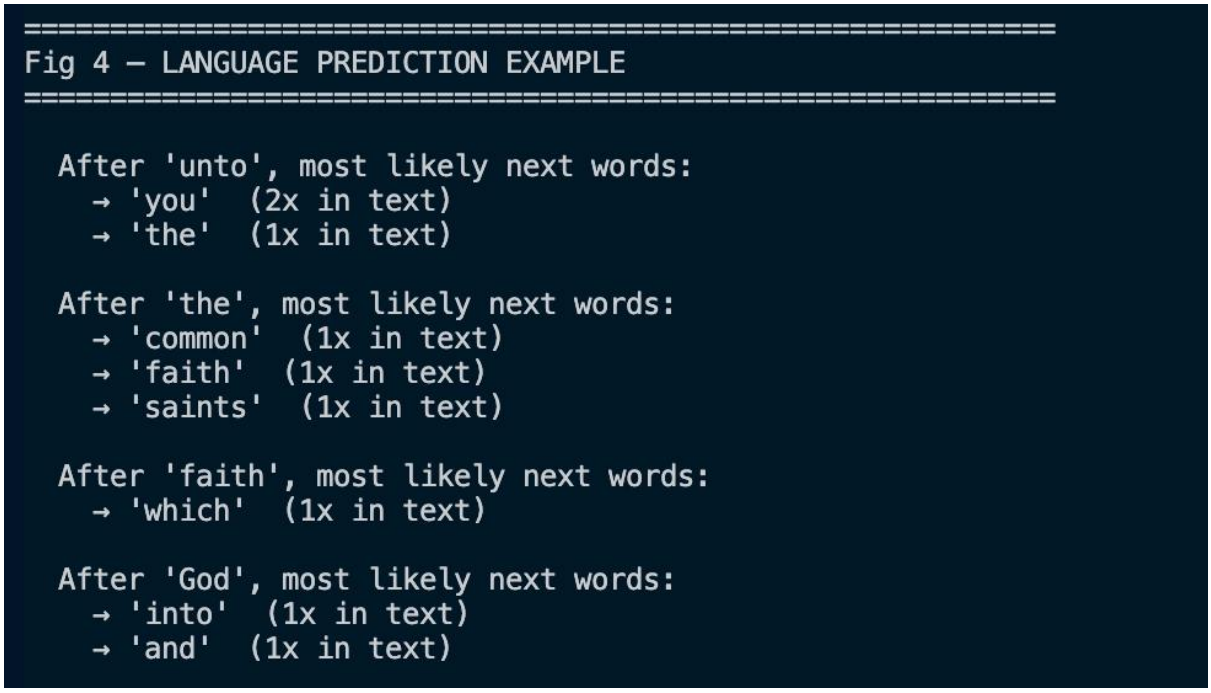


Fig 4: *Demonstrates simple language prediction using bigram frequencies. Given a word (e.g. 'unto'), the model looks at all bigrams starting with that word and returns the most likely word to follow.*

Fig 5: Sentence Analysis Output

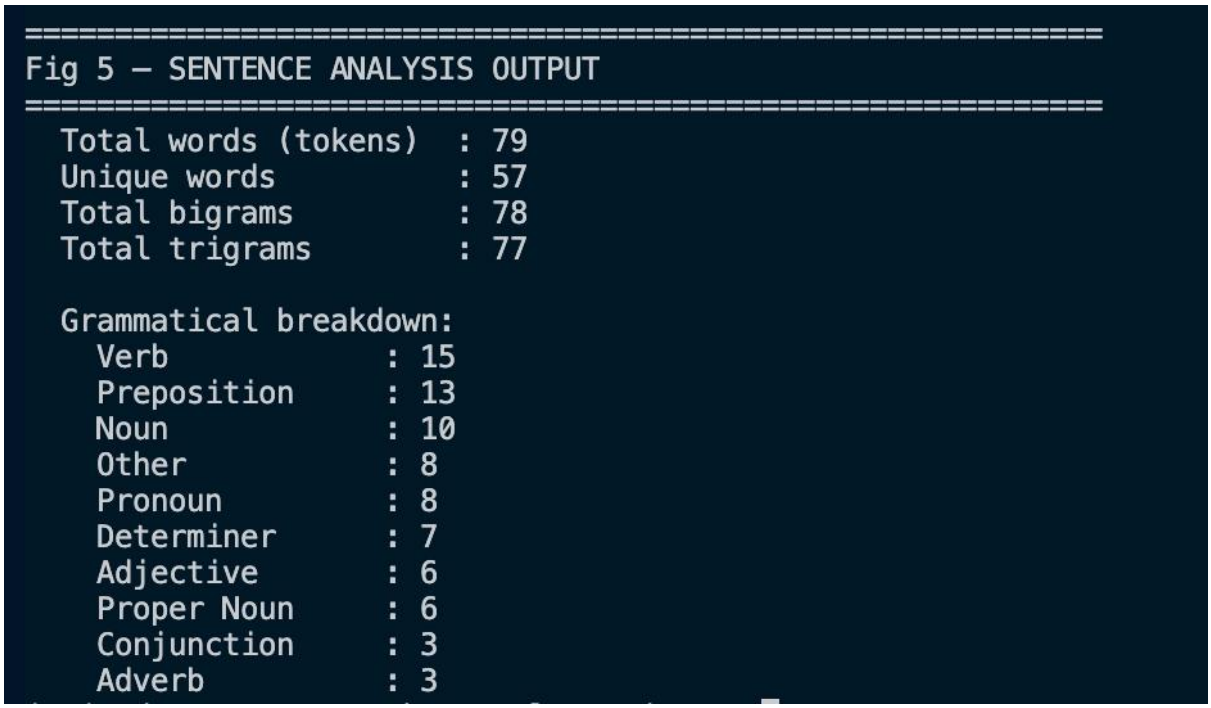


Fig 5: *Provides a complete structural breakdown of the text*

Student Reflection

This week introduced N-gram models and Part-of-Speech tagging using Python and NLTK. N-grams are sequences of consecutive words that reveal language patterns; bigrams capture two-word pairs while trigrams capture three-word sequences. I used these to build a simple next-word predictor on scripture from the Book of Jude. POS tagging then labelled each word with its grammatical role, showing which words are nouns, verbs, or adjectives. The main challenge was understanding the Penn Treebank tag codes, so I mapped them to plain English labels.

WEEK 3: HIDDEN MARKOV MODELS AND SEQUENCE LABELING

Sequence Prediction and Hidden State Analysis

Fig 1: Sequence Labeling Code

```
============================================================
Fig 1 — SEQUENCE LABELING CODE
============================================================

  # Our own HMM — trains transition + emission probabilities from MACULA data
  tagger = BiblicalHMM()
  tagger.train(train_sentences)

  # Each sentence is a list of (english_gloss, pos_tag) tuples
  # e.g. [('jude', 'NNP'), ('servant', 'NN'), ('christ', 'NNP'), ...]

  # Viterbi decoding finds the most likely tag sequence for new text
  tokens = word_tokenize(sentence)
  tagged = tagger.tag(tokens)

Training HMM on MACULA Greek NT... done.
```

Fig 1: *Shows the code for our custom HMM tagger (BiblicalHMM). NLTK's built-in HMM tagger has a known numerical overflow bug on Python 3.9+.*

Fig 2: Hidden Markov Model Output

```
============================================================
Fig 2 — HIDDEN MARKOV MODEL OUTPUT
============================================================
Input:
  "Jude, the servant of Jesus Christ, and brother of James, to them that are sanctified by God the Father, and preserved in Jesus Christ, and ca
lled: Mercy unto you, and peace, and love, be multiplied."

Word                HMM Tag     Label
------------------------------------------------
  Jude              CC          Conjunction
  ,                 VB          Verb
  the               DT          Determiner
  servant           NN          Noun
  of                IN          Preposition
  Jesus             NNP         Proper Noun
  Christ            NNP         Proper Noun
  ,                 NNP         Proper Noun
  and               CC          Conjunction
  brother           NN          Noun
  of                IN          Preposition
  James             NNP         Proper Noun
  ,                 VB          Verb
  to                IN          Preposition
  them              PRP         Pronoun
  that              CC          Conjunction
  are               VB          Verb
  sanctified        VB          Verb
  by                IN          Preposition
  God               NN          Noun
  the               DT          Determiner
  Father            NN          Noun
  ,                 PRP         Pronoun
  and               CC          Conjunction
  preserved         VB          Verb
  in                IN          Preposition
  Jesus             NNP         Proper Noun
  Christ            NNP         Proper Noun
  ,                 NNP         Proper Noun
  and               CC          Conjunction
  called            VB          Verb
  :                 DT          Determiner
  Mercy             NN          Noun
  unto              IN          Preposition
  you               PRP         Pronoun
  ,                 VB          Verb
  and               CC          Conjunction
  peace             NN          Noun
  ,                 PRP         Pronoun
  and               CC          Conjunction
  love              VB          Verb
  ,                 PRP         Pronoun
  be                VB          Verb
  multiplied        VB          Verb
  .                 IN          Preposition
```

Fig 2: *Shows the HMM output on the first verse of Jude.*

Fig 3: Named Entity Recognition Example (NLTK)

```
============================================================
Fig 3 — NAMED ENTITY RECOGNITION EXAMPLE
============================================================
Input: "Jude, the servant of Jesus Christ, and brother of James, to them that are sanctified by God the Father, and preserved in Jesus Christ, a
nd called: Mercy unto you, and peace, and love, be multiplied."

  [GPE]  →  'Jude'
  [PERSON]  →  'Jesus Christ'
  [PERSON]  →  'James'
  [GPE]  →  'Father'
  [GPE]  →  'Jesus Christ'
```

Fig 3: *Shows NER applied to the first verse using NLTK's ne_chunk().*

Fig 4: Sentence Labeling Results

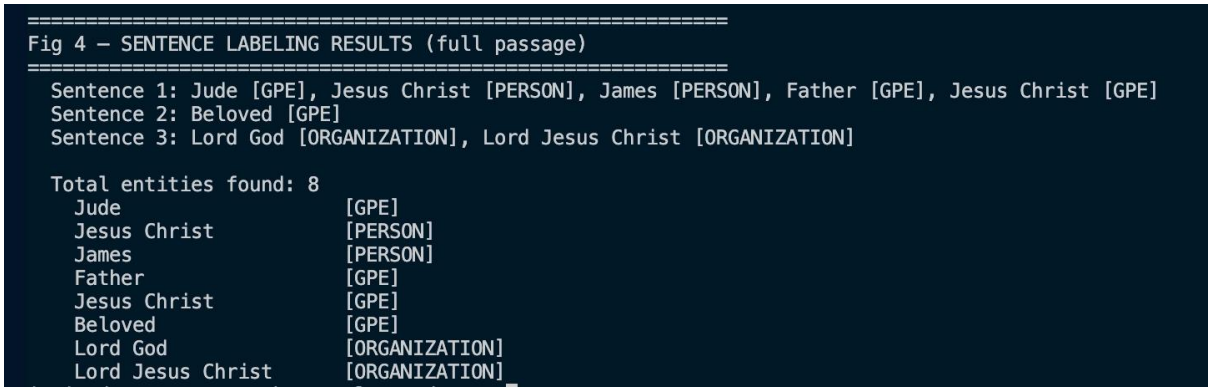


Fig 4: *Shows NER across Jude 1:1-7. Lord Jesus Christ and Lord God are classified as ORGANIZATION rather than PERSON, a domain mismatch limitation, not a bug.*

Fig 5: Dataset Used for Training or Testing

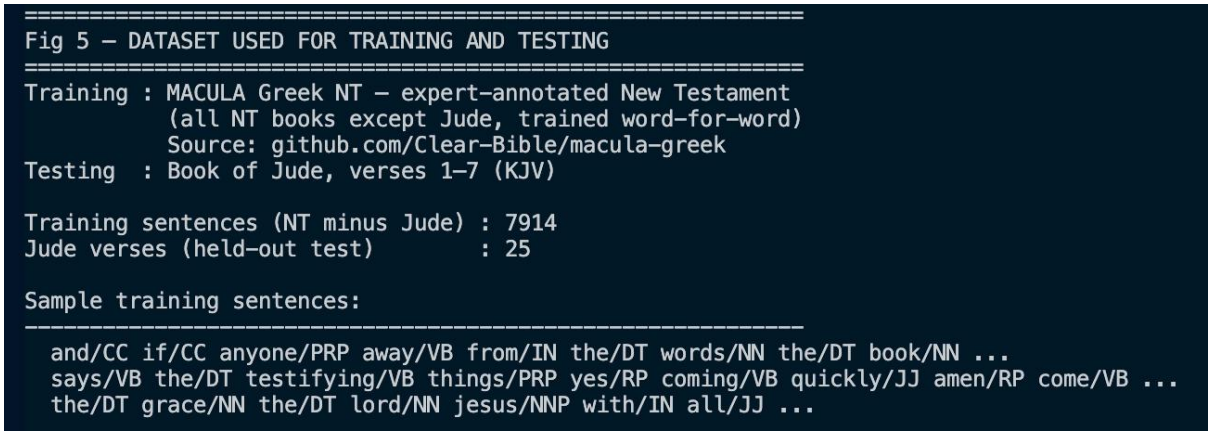


Fig 5: *Shows the datasets used, MACULA Greek NT is expert-annotated NT text with verified POS tags, trained on all NT books (7,914 verses) with Jude held out as the test set.*

Bonus: Improved NER: spaCy vs NLTK

NLTK's ne_chunk was trained on news text, causing Jude and Jesus Christ to be labelled as GPE (place), and Lord Jesus Christ as ORGANIZATION. To demonstrate a better approach, the same passage was run through spaCy's en_core_web_sm: a neural NER model trained on the OntoNotes 5 corpus (~1 million words, diverse genres).

Key Observations: spaCy correctly identifies Jude, Jesus Christ, and James as PERSON.

Both models miss titles like Lord God and Lord Jesus Christ too domain-specific for any modern-trained NER model.

Student Reflection

This week introduced Hidden Markov Models and sequence labeling. Training on MACULA Greek NT expert-annotated biblical text produced correctly varied POS tags. For NER, NLTK's ne_chunk misclassified Jude as a place and Lord Jesus Christ as an organisation because it was trained on news text. Replacing it with spaCy corrected Jude and Jesus Christ to PERSON. Both models missed titles like Lord God, showing that even better models have limits when applied to domain-specific ancient text.

WEEK 4: SYNTACTIC AND SEMANTIC ANALYSIS

Dependency Parsing and Semantic Similarity

Fig 1: Dependency Parsing Example

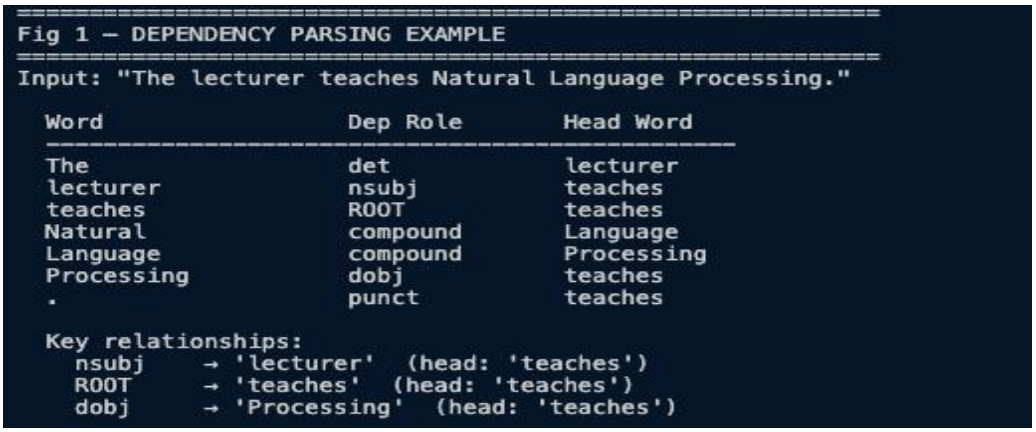


Fig 1: *shows the output of spaCy's dependency parser*

Fig 2: Dependency Parsing on Biblical Text

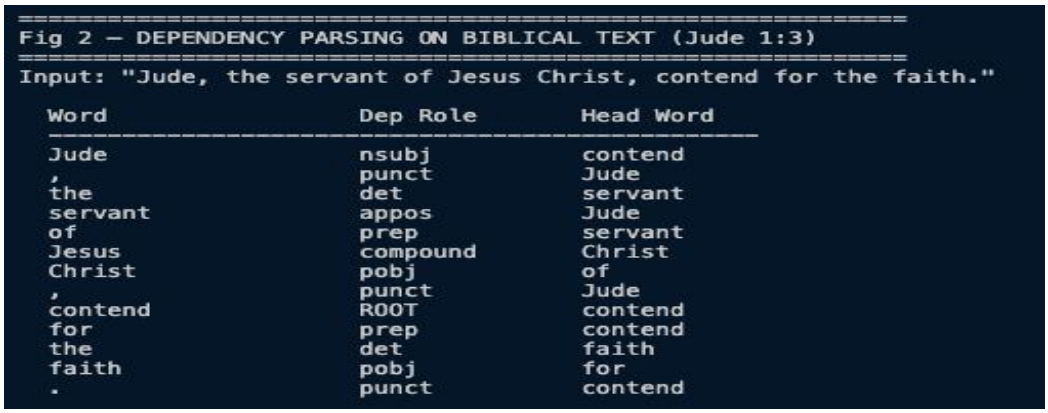


Fig 2: *applies the same parser to Jude 1:3*

Fig 3: Assignment 1 Dependency Analysis

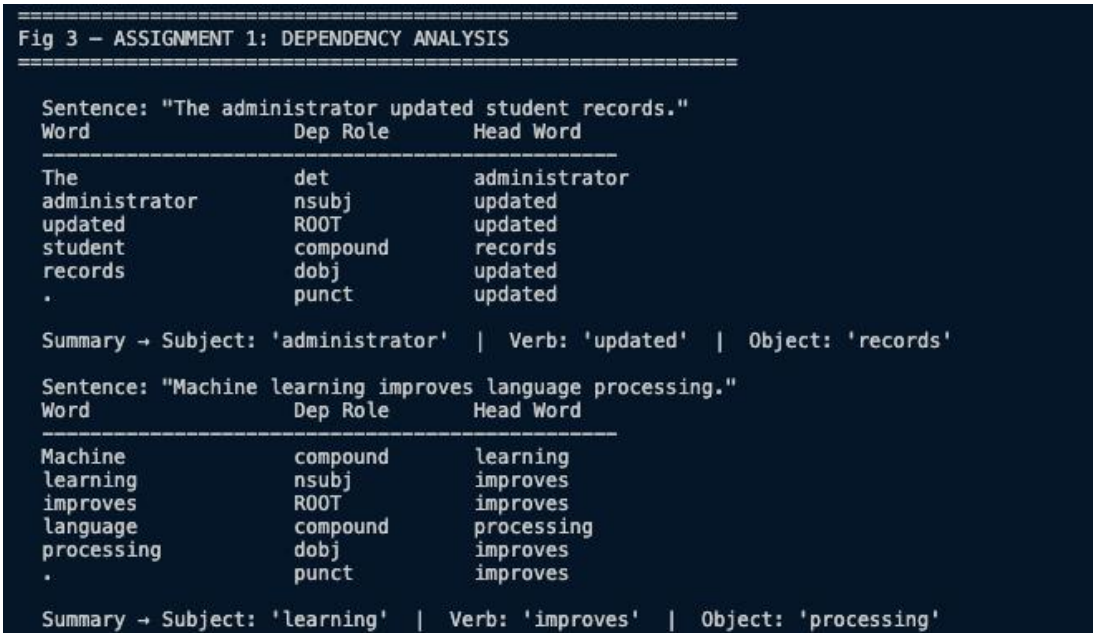


Fig 3: *Shows dependency analysis for two sentences*

Fig 4: Semantic Similarity Analysis

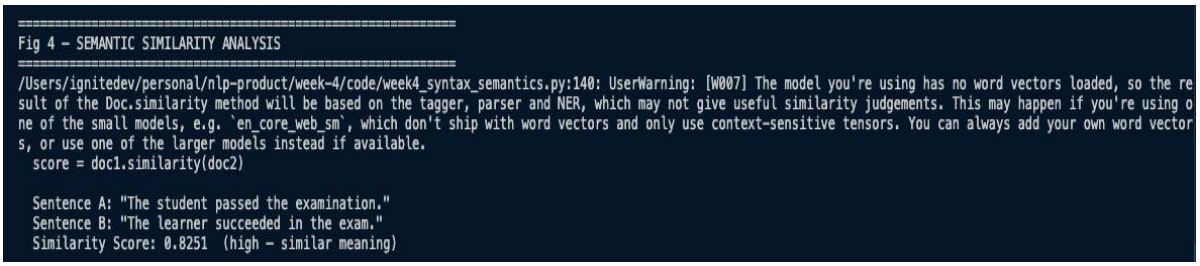


Fig 4: *Shows spaCy comparing 2 sentences*

Fig 5: Assignment 2 Similar vs Unrelated Sentence Pairs

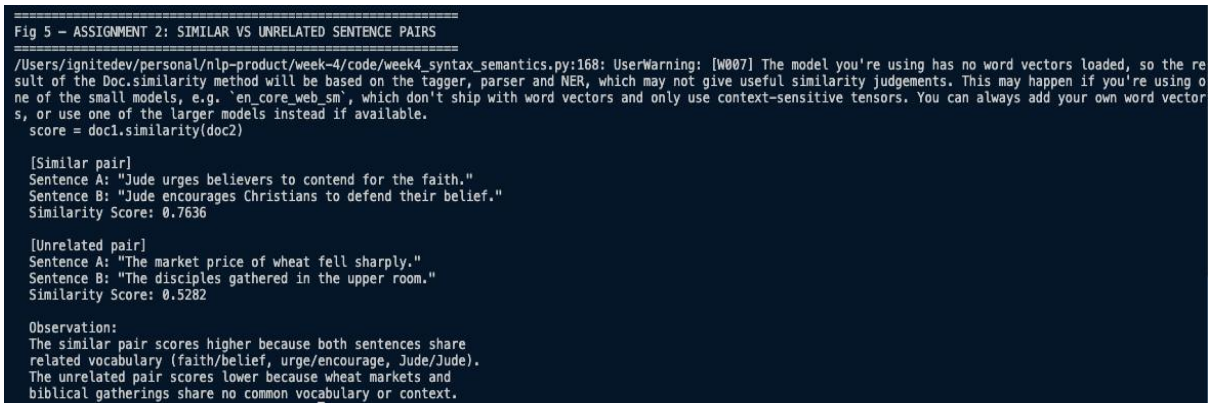


Fig 5: *Shows the contrast between two pairs of sentences*

Student Reflection

This week introduced dependency parsing and semantic similarity using spaCy. Dependency parsing reveals sentence structure by labelling each word with its grammatical role and linking it to the word it depends on, making it possible to extract subject, verb, and object automatically. Semantic similarity goes further by comparing meaning rather than exact words; two sentences can score highly even when phrased completely differently. Applying these tools to the Book of Jude highlighted a challenge: archaic KJV phrasing causes some dependency labels to differ from what a modern English parser expects.

WEEK 5: MINI NLP PROJECTS

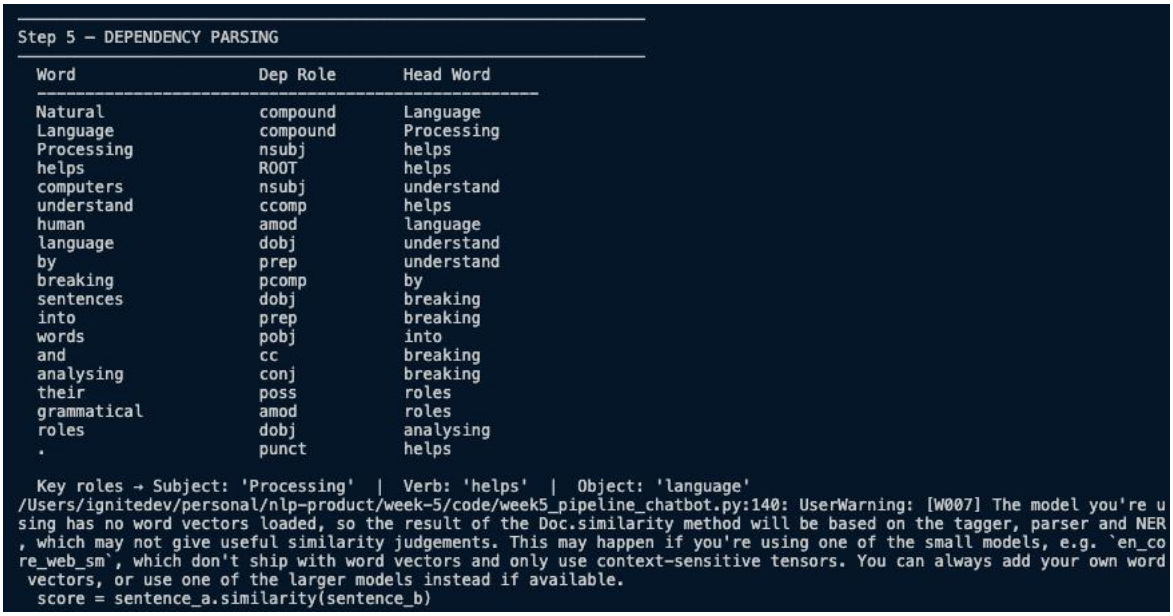
Complete NLP Pipeline and Mini Chatbot

Practical Task 1: Complete NLP Pipeline Output

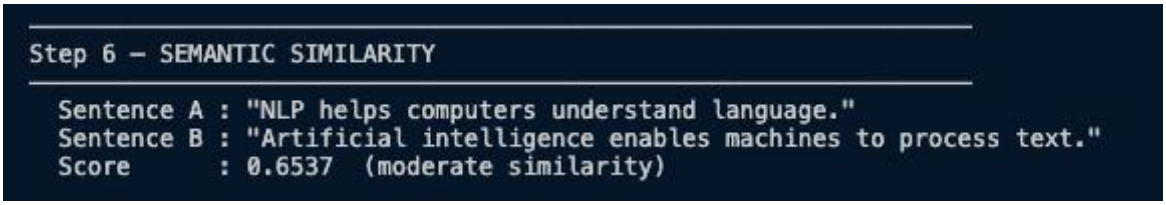
```
ignitedev@Ians-MacBook-Pro nlp-product % python3 week-5/code/week5_pipeline_chatbot.py
/Users/ignitedev/Library/Python/3.9/lib/python/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only
 supports OpenSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3
/urllib3/issues/3020
  warnings.warn(
============================================================
PRACTICAL TASK 1 — COMPLETE NLP PIPELINE
============================================================

Input text:
  "Natural Language Processing helps computers understand human language by breaking sentences into words and analysing
 their grammatical roles."

------------------------------------------------------------
Step 1 — TOKENIZATION
------------------------------------------------------------
  Total tokens  : 19
  Token list    : ['Natural', 'Language', 'Processing', 'helps', 'computers', 'understand', 'human', 'language', 'by',
'breaking', 'sentences', 'into', 'words', 'and', 'analysing', 'their', 'grammatical', 'roles', '.']

------------------------------------------------------------
Step 2 — STOPWORD REMOVAL
------------------------------------------------------------
  Before removal : 18 words
  After removal  : 14 words
  Filtered list  : ['Natural', 'Language', 'Processing', 'helps', 'computers', 'understand', 'human', 'language', 'brea
king', 'sentences', 'words', 'analysing', 'grammatical', 'roles']
```

```
------------------------------------------------------------
Step 3 — POS TAGGING
------------------------------------------------------------
  Word                 Tag      Label
  ----------------------------------------
  Natural              JJ       Adjective
  Language             NNP      Proper Noun
  Processing           NNP      Proper Noun
  helps                VBZ      Verb
  computers            NNS      Noun
  understand           VBP      Verb
  human                JJ       Adjective
  language             NN       Noun
  breaking             VBG      Verb
  sentences            NNS      Noun
  words                NNS      Noun
  analysing            VBG      Verb
  grammatical          JJ       Adjective
  roles                NNS      Noun

  Grammatical breakdown:
    Noun            : 5
    Verb            : 4
    Adjective       : 3
    Proper Noun     : 2
```

```
------------------------------------------------------------
Step 4 — N-GRAMS
------------------------------------------------------------
  Total bigrams  : 13
  Total trigrams : 12
  Sample bigrams : [('Natural', 'Language'), ('Language', 'Processing'), ('Processing', 'helps'), ('helps', 'computers'
), ('computers', 'understand')]
  Sample trigrams: [('Natural', 'Language', 'Processing'), ('Language', 'Processing', 'helps'), ('Processing', 'helps',
 'computers')]
```



```
Step 5 — DEPENDENCY PARSING

  Word                 Dep Role       Head Word
  ------------------------------------------------
  Natural              compound       Language
  Language             compound       Processing
  Processing           nsubj          helps
  helps                ROOT           helps
  computers            nsubj          understand
  understand           ccomp          helps
  human                amod           language
  language             dobj           understand
  by                   prep           understand
  breaking             pcomp          by
  sentences            dobj           breaking
  into                 prep           breaking
  words                pobj           into
  and                  cc             breaking
  analysing            conj           breaking
  their                poss           roles
  grammatical          amod           roles
  roles                dobj           analysing
  .                    punct          helps

  Key roles → Subject: 'Processing'  |  Verb: 'helps'  |  Object: 'language'
/Users/ignitedev/personal/nlp-product/week-5/code/week5_pipeline_chatbot.py:140: UserWarning: [W007] The model you're u
sing has no word vectors loaded, so the result of the Doc.similarity method will be based on the tagger, parser and NER
, which may not give useful similarity judgements. This may happen if you're using one of the small models, e.g. `en_co
re_web_sm`, which don't ship with word vectors and only use context-sensitive tensors. You can always add your own word
 vectors, or use one of the larger models instead if available.
  score = sentence_a.similarity(sentence_b)
```



```
Step 6 — SEMANTIC SIMILARITY

  Sentence A : "NLP helps computers understand language."
  Sentence B : "Artificial intelligence enables machines to process text."
  Score      : 0.6537  (moderate similarity)
```

The screenshots above shows the complete NLP pipeline outputs from the terminal

Fig 2: Chatbot Demo Output

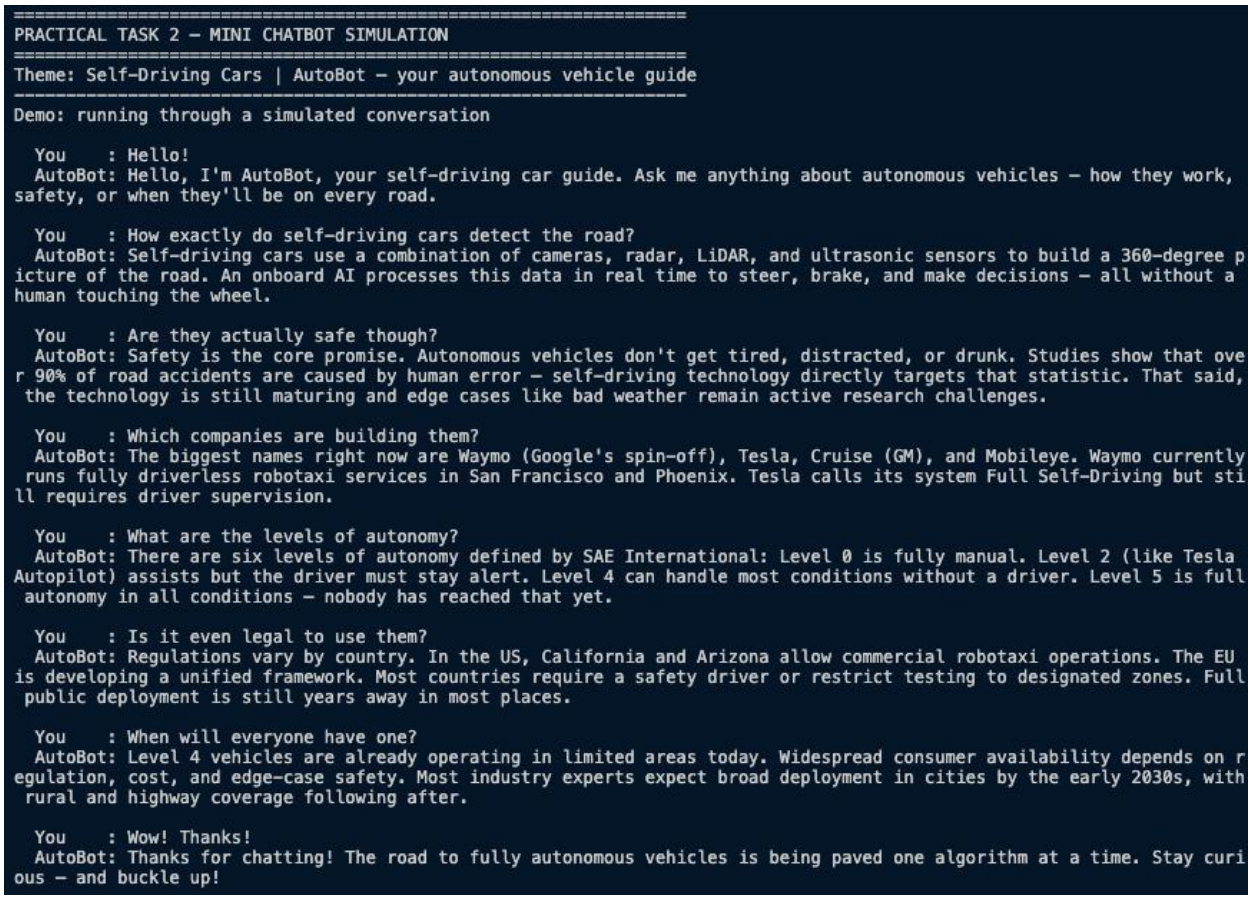
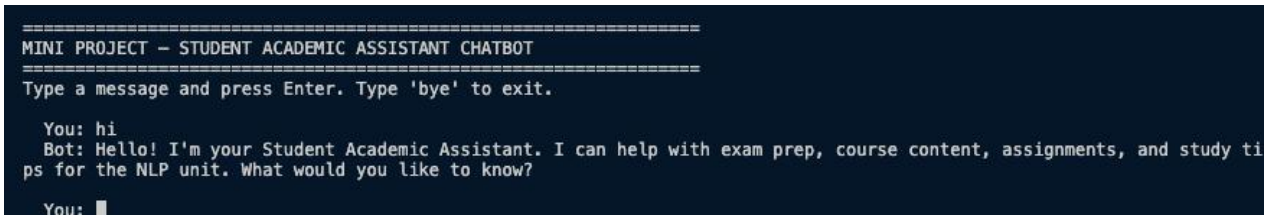


Fig 2 shows the chatbot simulation

Fig 3: Interactive Chatbot Session



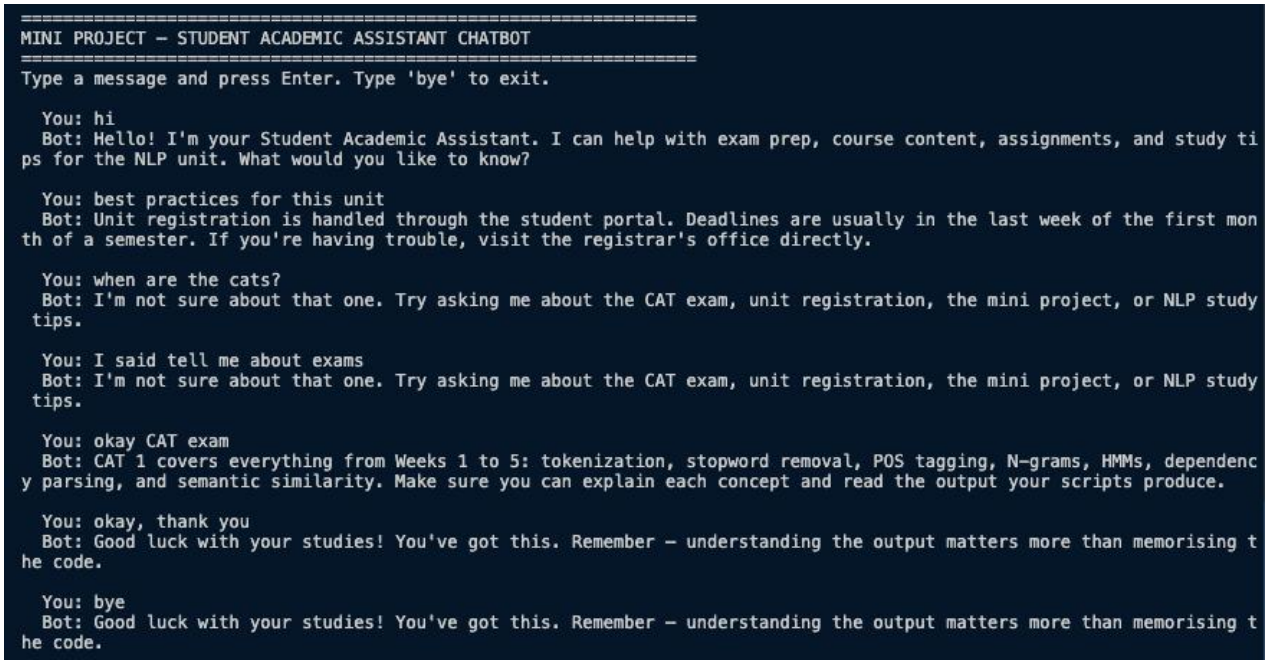
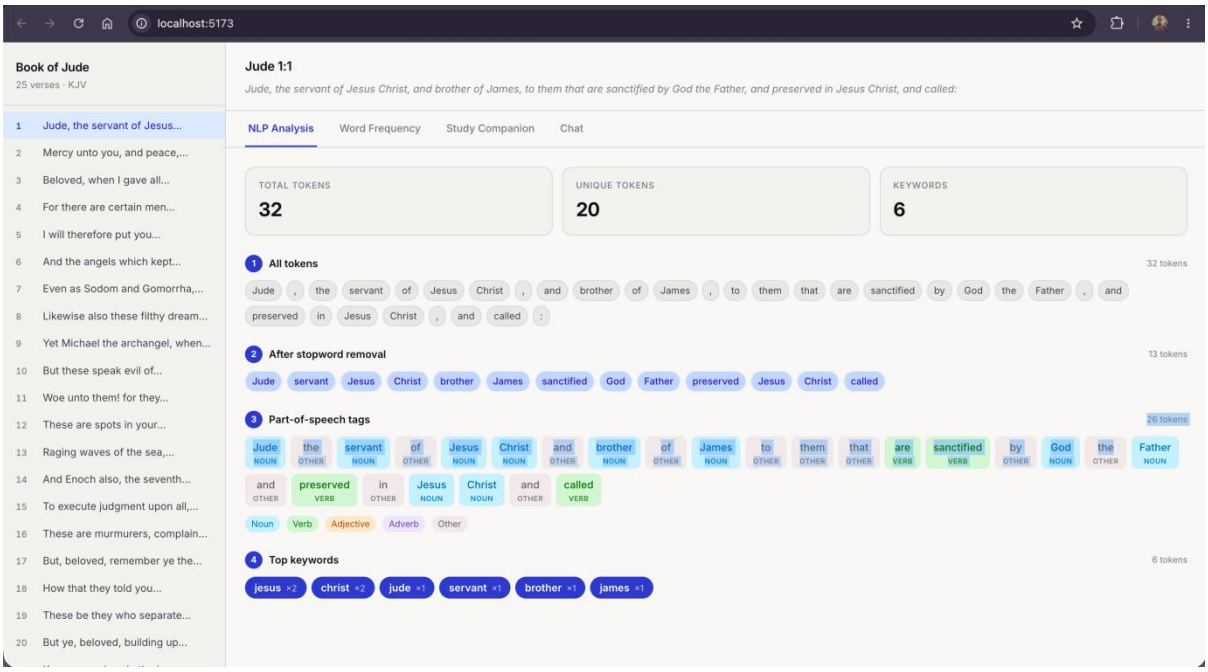


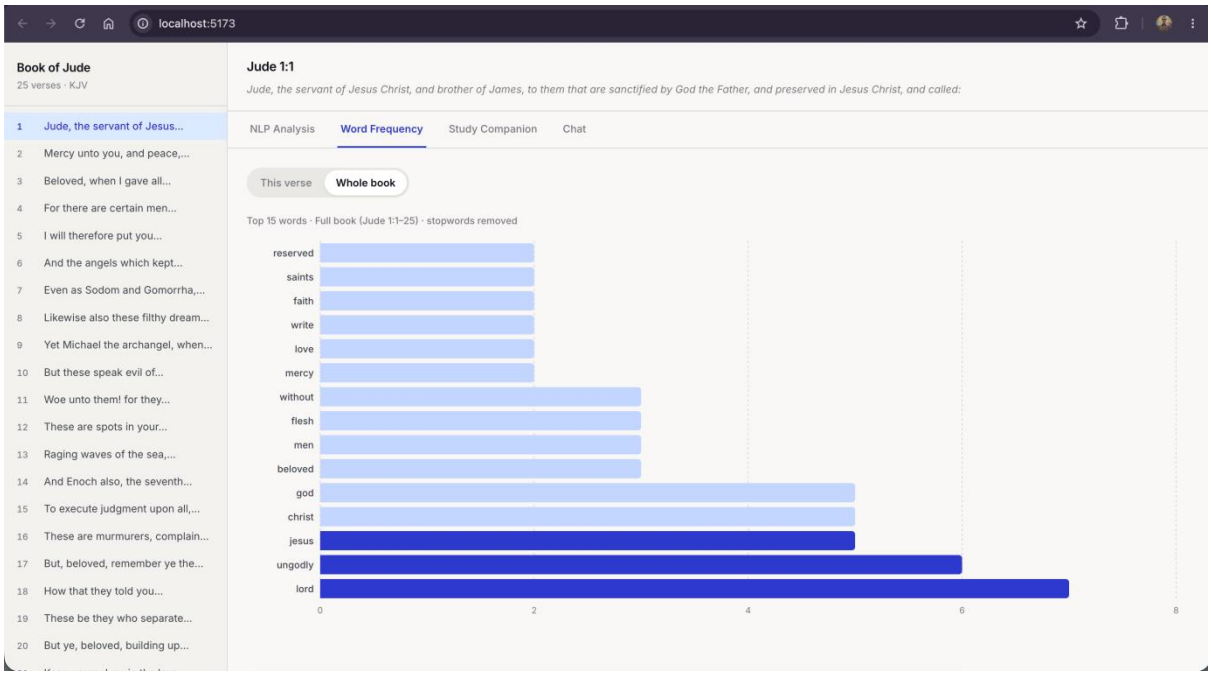
Fig 3 shows the chatbot running in interactive mode where the user types their own messages to the chatbot

Mini Project: AI Bible Study Companion (jude-companion)

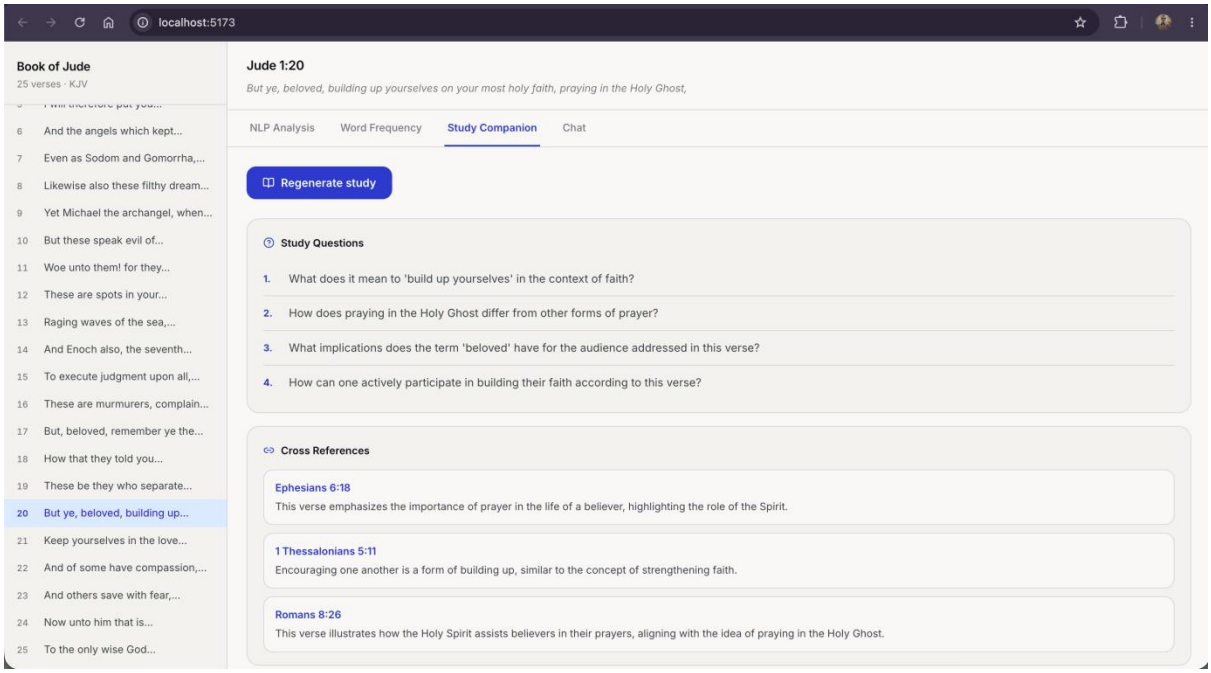
AI Bible Study Companion is a web application that applies NLP techniques to the Book of Jude (KJV)



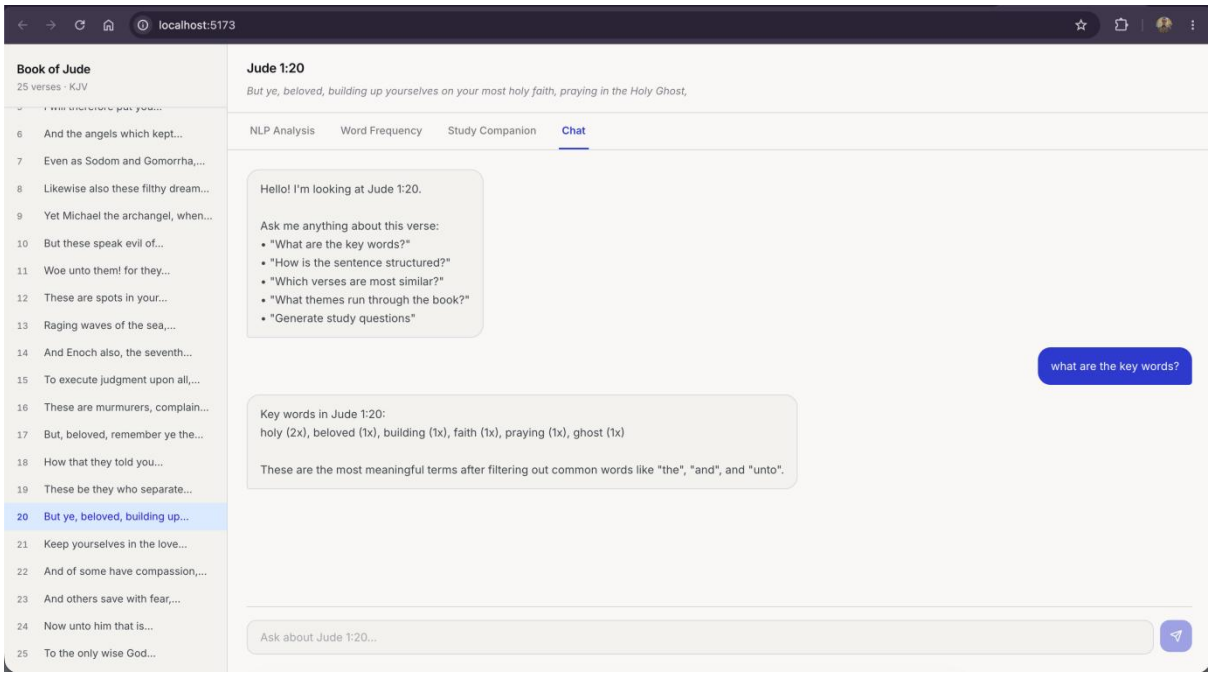
The NLP Analysis tab shows the full processing pipeline applied to the selected verse



The Word Frequency tab renders a horizontal bar chart of the most repeated meaningful words



The Study Companion tab uses GPT-4o-mini to generate AI-powered study material for the selected verse.



The Chat tab provides a conversational interface to the NLP backend

Student Reflection

Week 5 brought together every technique from the course into a single working pipeline. Running tokenization, stopword removal, POS tagging, n-grams, dependency parsing, and semantic similarity. The mini chatbot showed how rule-based NLP systems work at a basic level, matching user input to intents using keyword detection.

WEEK 6: WORD EMBEDDINGS AND DISTRIBUTED REPRESENTATIONS

Practical task 1: Train a word2vec model

```
======================================================================
PRACTICAL TASK 1 — TRAIN A WORD2VEC MODEL
======================================================================

  Vocabulary size : 11 words
  Vector size     : 50 dimensions

  Vector for 'nlp' (first 8 of 50 values):
    [-0.016  0.009 -0.008  0.002  0.017 -0.009  0.009 -0.014] ...

  Words most similar to 'nlp':
    love         → 0.1249
    modern       → 0.0740
    learning     → 0.0424
```

Training a word2vec model

Practical task 2: Similarity analysis

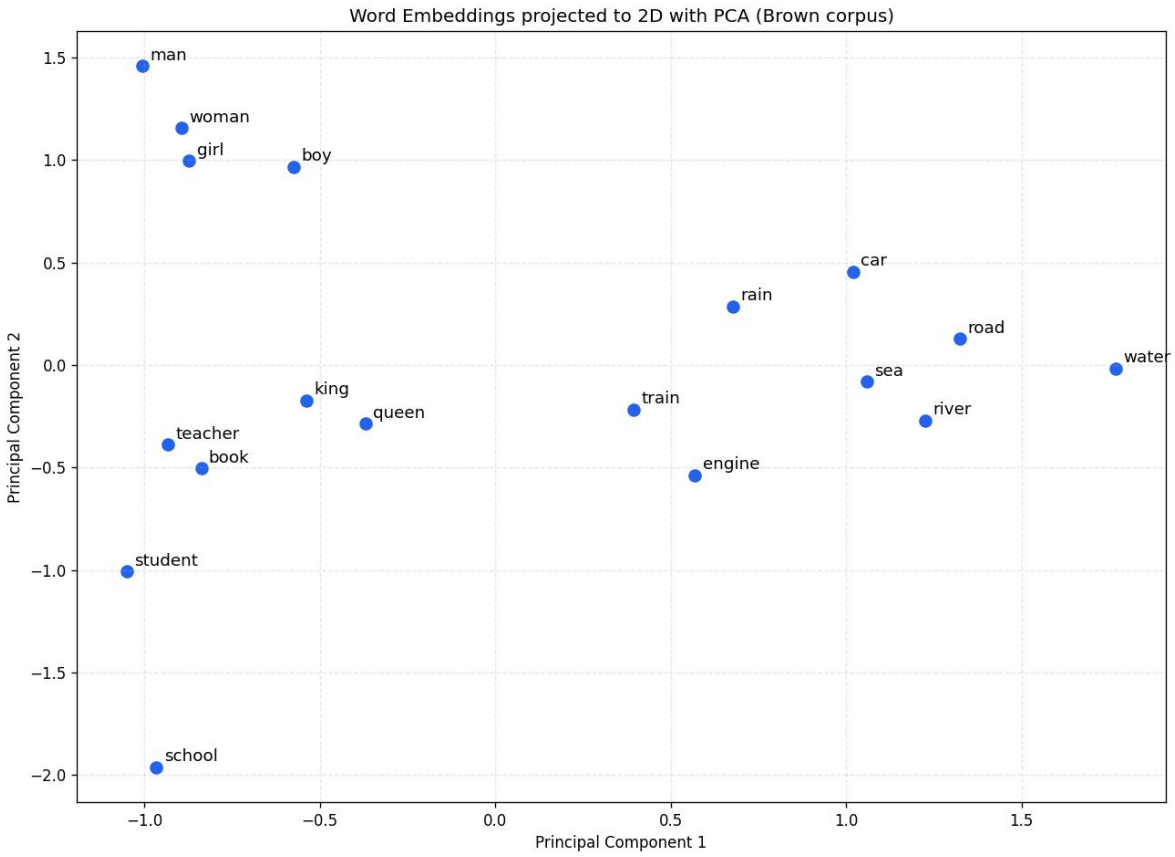
```
======================================================================
PRACTICAL TASK 2 — SIMILARITY ANALYSIS (10 WORDS)
======================================================================

  Word         Nearest neighbour    Cosine score
  ------------------------------------------------
  man          woman                0.8141
  woman        girl                 0.9057
  school       college              0.8148
  money        excuse               0.7976
  city         yankees              0.8273
  water        hot                  0.8158
  music        jazz                 0.8199
  war          civil                0.7975
  science      historical           0.8968
  government   federal              0.8838

  Direct similarity('man', 'woman') = 0.8141

======================================================================
WORD ANALOGY — king - man + woman ≈ ?
======================================================================
  king - man + woman is closest to:
    richard      → 0.8064
    smith        → 0.8056
    queen        → 0.8038
```

Word2Vec (Skip-Gram, 100-dim) trained on the Brown corpus, The model learned real semantic relationships

Practical task 3: One-hot encoding vs word embeddings

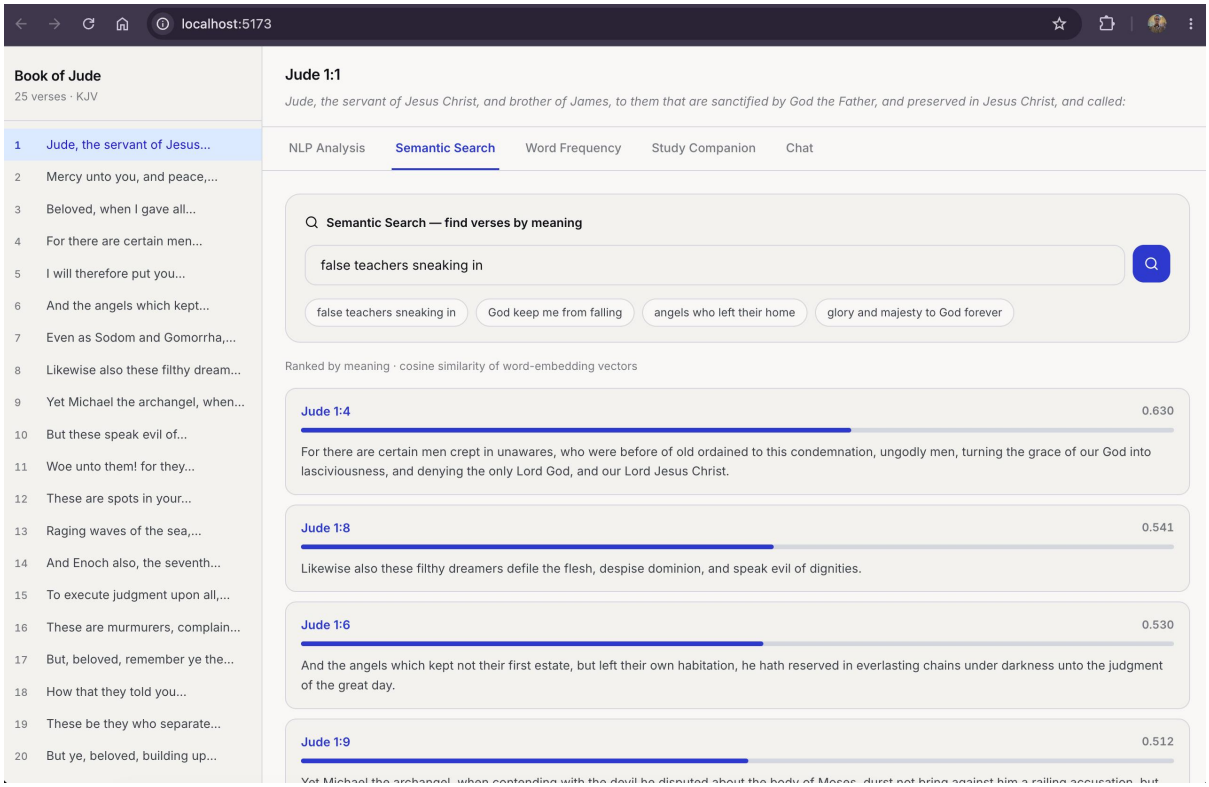
```
======================================================================
PRACTICAL TASK 3 — ONE-HOT ENCODING vs WORD EMBEDDINGS
======================================================================

  One-Hot Encoding (each word is an isolated 1):
    cat      → [1, 0, 0]
    dog      → [0, 1, 0]
    animal   → [0, 0, 1]

  Word Embedding (dense, learned from context — first 6 dims):
    cat      → [ 0.021  0.225 -0.103  0.2   -0.05  -0.26 ] ...
    dog      → [-0.065  0.069 -0.085  0.218 -0.044 -0.108] ...
    animal   → [-0.047  0.067 -0.018  0.046 -0.143 -0.271] ...

  COMPARISON REPORT
  ------------------------------------------------------------
  Feature                   One-Hot Encoding  Word Embeddings
  ------------------------------------------------------------
  Size                      Vocab length      Fixed & compact
  Meaning representation    None              Captures meaning
  Efficiency                Sparse / wasteful Dense / efficient
  Semantic understanding    No                Yes
  ------------------------------------------------------------
  Conclusion: One-Hot vectors grow with the vocabulary and treat every
  word as unrelated. Embeddings are compact, fixed-size, and place words
  with similar meaning close together in vector space.
```

One-Hot Encoding versus Word Embeddings: sparse isolated vectors with no meaning, contrasted with dense vectors that capture semantic relationships



100-dim Brown-corpus embeddings projected to 2D with PCA to depict similarity

New concept: Embedding-powered semantic search engine



Concept applied to the AI Bible Study Companion: an embedding-powered semantic search. Each verse is encoded as a document embedding (the average of its word vectors); the query is embedded the same way and verses are ranked by cosine similarity. The query 'false teachers sneaking in' returns Jude 1:4 despite sharing no words with it matching by meaning rather than keywords.

Student Reflection

This week introduced word embeddings. Earlier methods like One-Hot Encoding treat every word as an isolated symbol, producing large sparse vectors with no sense of meaning. Word embeddings instead represent each word as a compact dense vector learned from context, so words used in similar ways end up close together. As the creative task, I applied embedding-powered semantic search engine to my main project by adding a semantic search feature to the AI Bible Study Companion: a query like "false teachers sneaking in" correctly returns Jude 1:4 about men who "crept in unawares", even though they share no words, because the match is made by meaning rather than keywords.

WEEK 7: NEURAL LANGUAGE MODELS AND DEEP LEARNING FOR NLP

Fig 1: TensorFlow installation

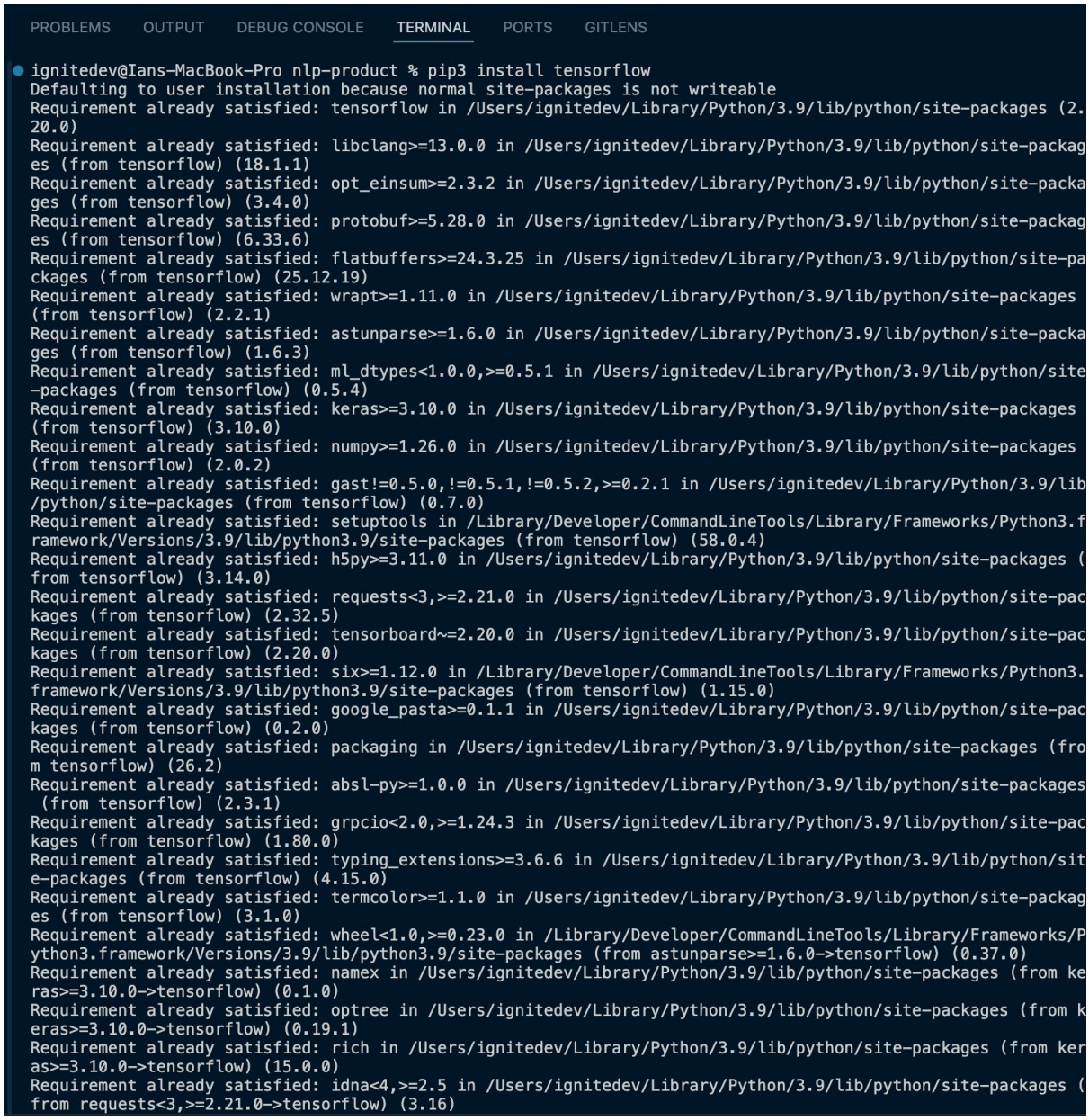


Fig 1: Shows TensorFlow being installed using pip3

Fig 2: Neural Language Model Training on Jude's Corpus

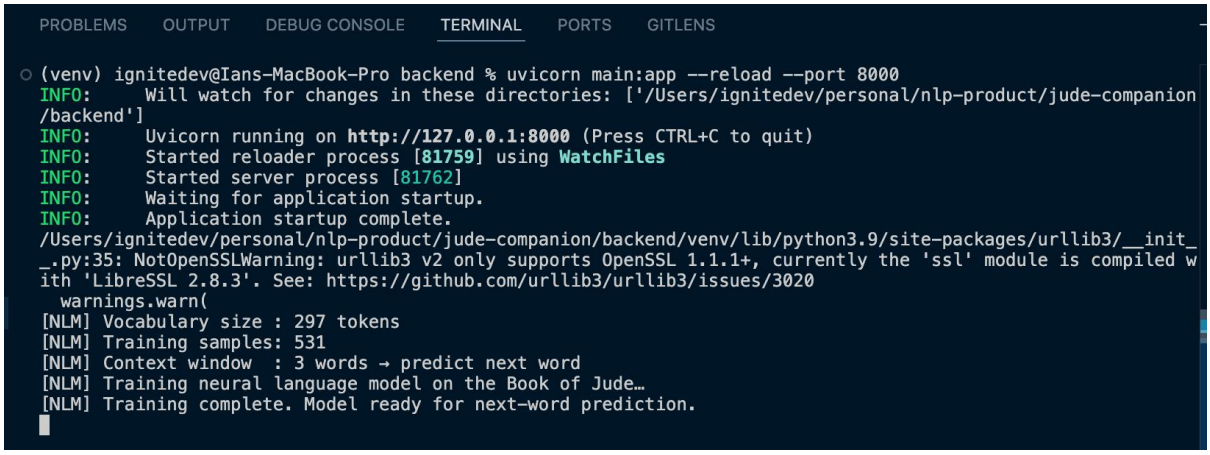


Fig 2: Shows the neural language model training at server startup.

Fig 3: Tokenizer Explorer - Word Index and Integer Sequences

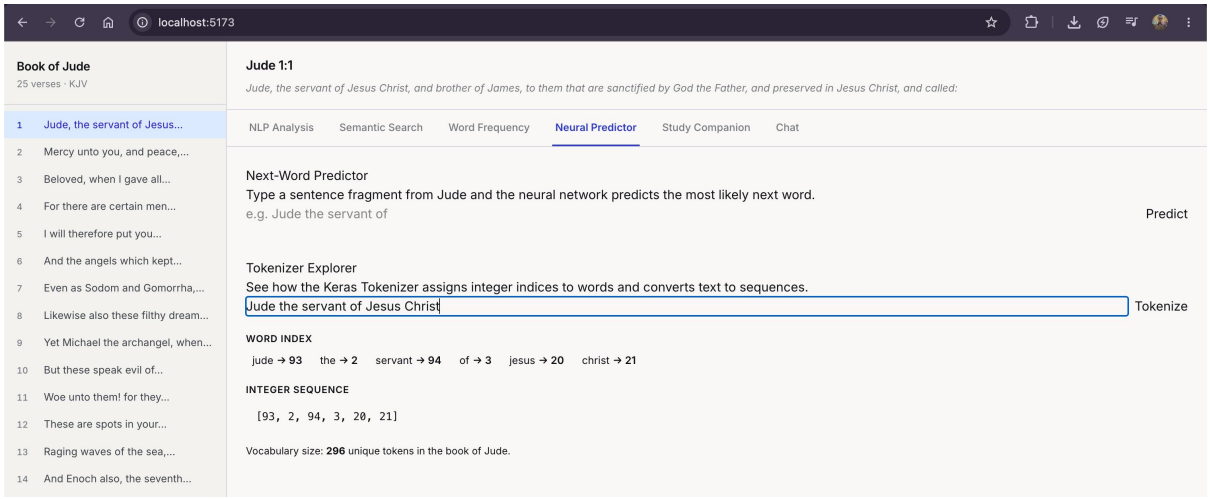


Fig 3: Shows the Tokenizer Explorer panel in the Neural Predictor tab

Fig 4: Next-Word Predictions with Confidence Scores

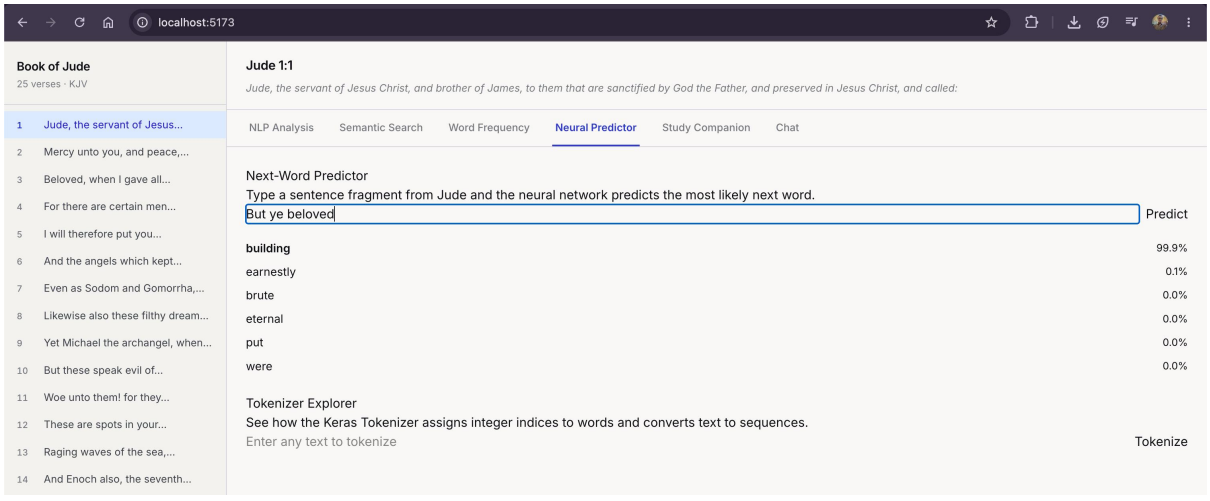


Fig 4: Shows the Neural Predictor accepting the fragment "But ye beloved" and returning the top predicted next words ranked by softmax confidence.

Fig 5: Neural Predictor Tab Full View

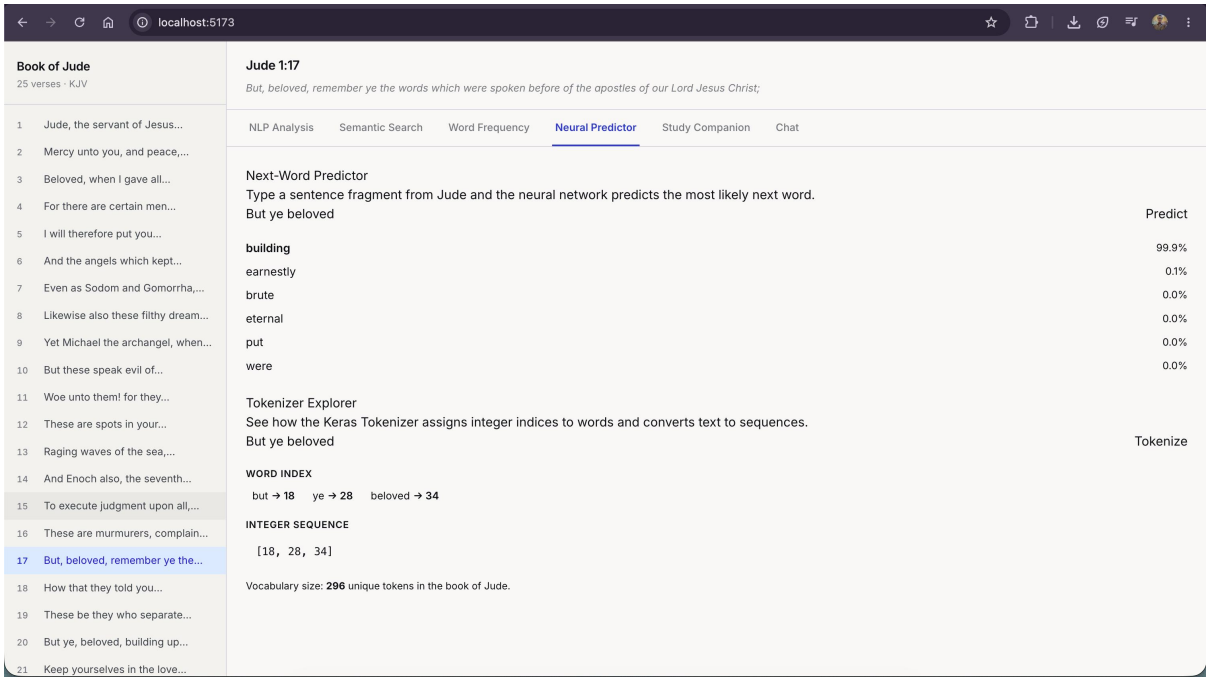


Fig 5: The complete Neural Predictor tab added to the AI Bible Study Companion

Student Reflection

This week introduced Neural Language Models and applied them to the AI Bible Study Companion project. A Keras neural network was trained on the Book of Jude using a 3-word sliding context window and a Softmax output layer to predict the next word with confidence scores. Unlike the N-gram model from Week 2, which counts word occurrences, the neural model uses an Embedding layer to learn word relationships, building directly on Week 6's word embeddings. A Neural Predictor tab was added to the frontend, enabling next-word prediction and live tokenizer exploration within the project.

WEEK 8: NEURAL LANGUAGE MODELS AND DEEP LEARNING FOR NLP

Fig 1: Installing Transformers and PyTorch

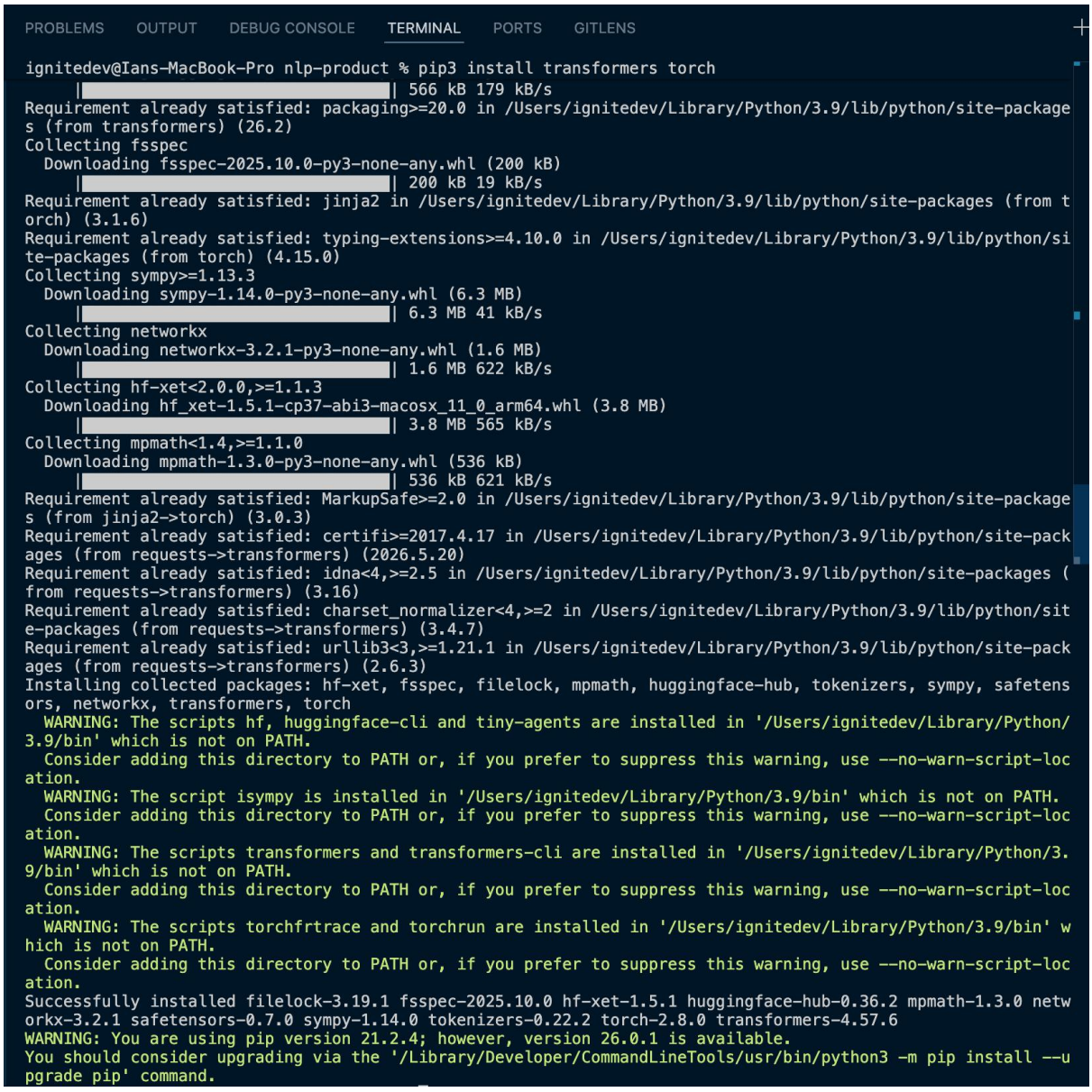


Fig 1: Shows the installation of the Hugging Face Transformers library and PyTorch using pip3

Fig 2: DistilBERT Pipeline Loading at Server Startup

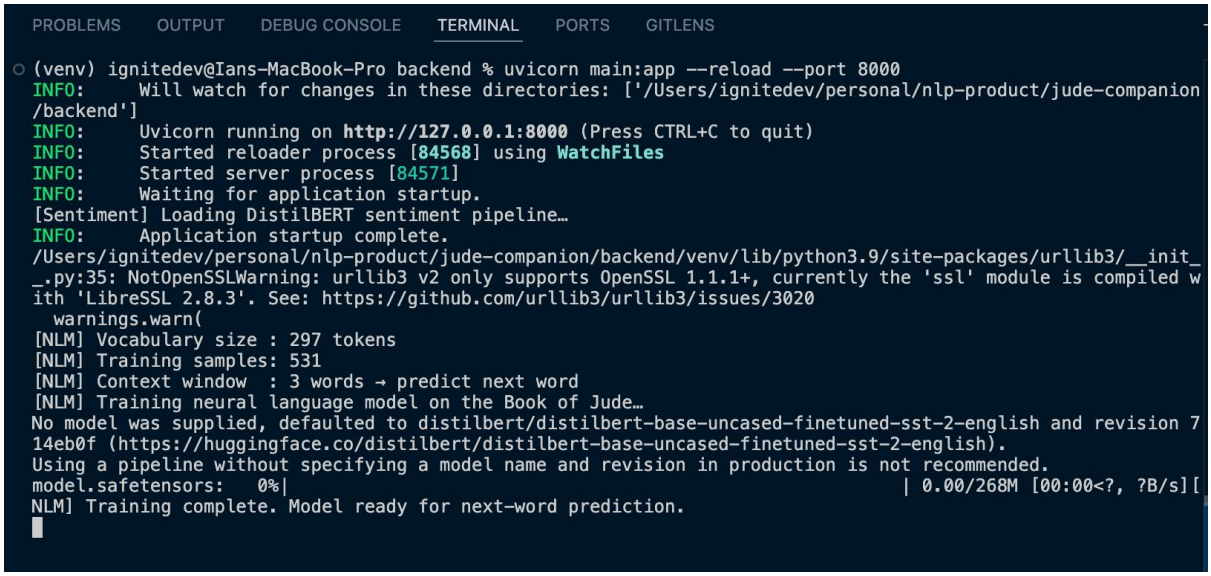


Fig 2: Shows the FastAPI backend loading the DistilBERT sentiment analysis pipeline at startup in a background thread.

Fig 3: Sentiment Analysis - Single Verse View

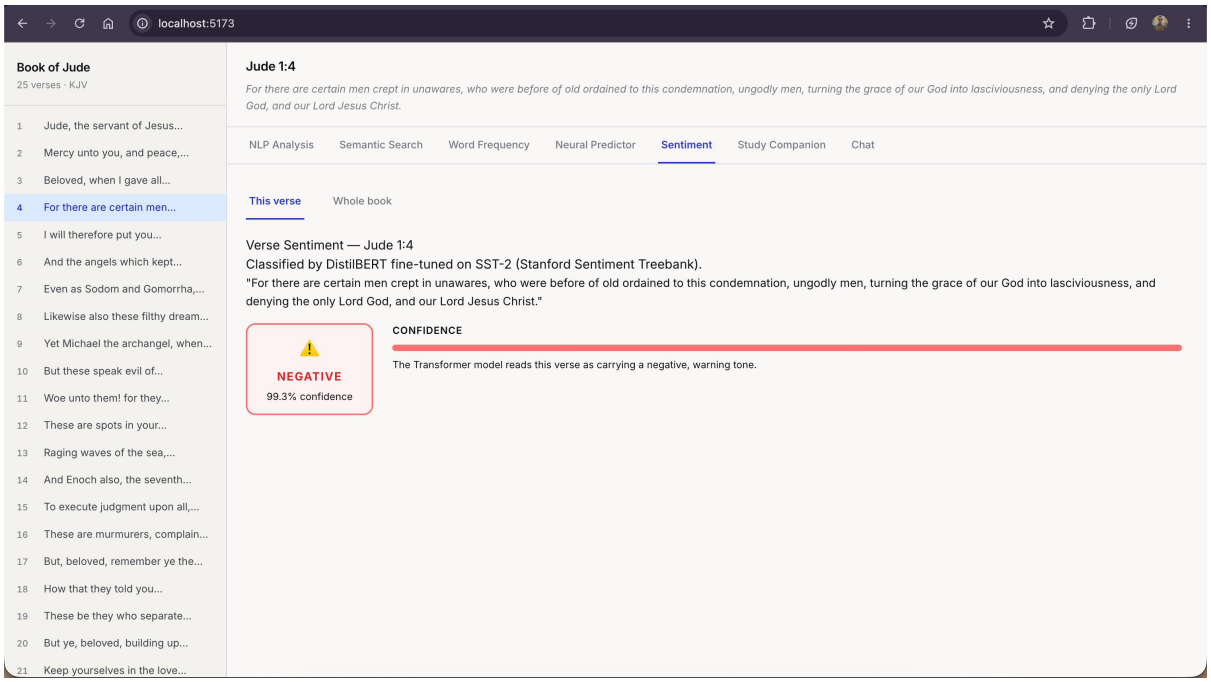


Fig 3: The Sentiment tab classifying a single verse from the Book of Jude using DistilBERT. The model returns a POSITIVE or NEGATIVE label with a confidence score displayed as a badge and confidence bar.

Fig 4: Sentiment Analysis - Whole Book View

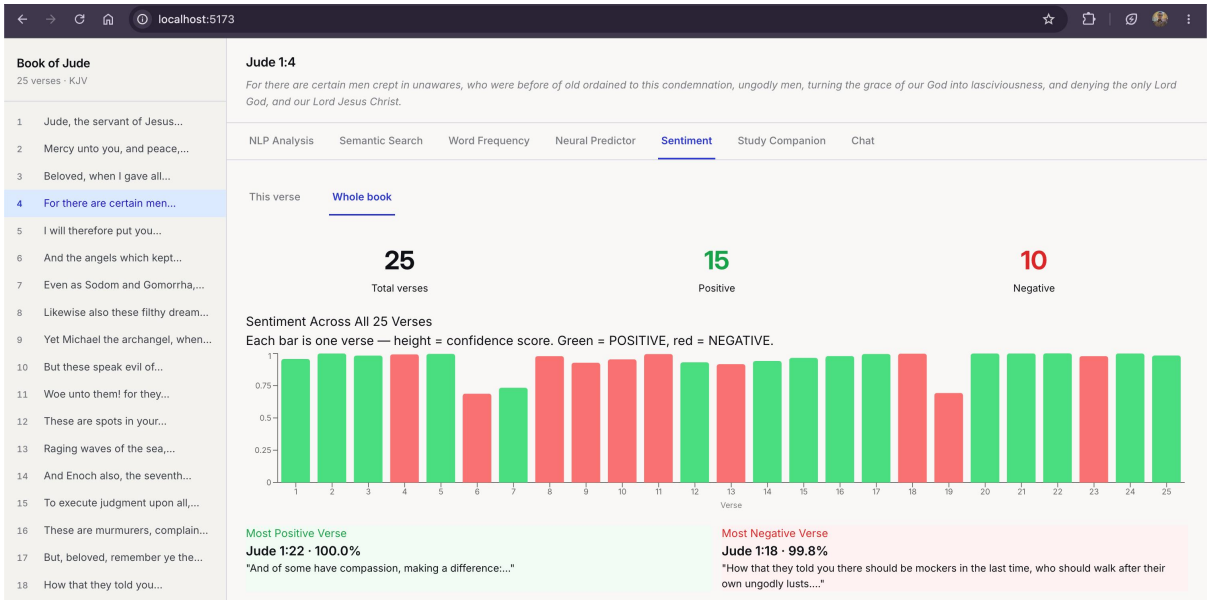


Fig 4: The whole book analysis plots sentiment confidence for all 25 verses as a bar chart — green for positive, red for negative.

Student Reflection

This week introduced Transformer models and the attention mechanism. Unlike the neural language model from Week 7 which was trained from scratch on Jude's small corpus, DistilBERT is pre-trained on millions of documents and can be applied directly via a pipeline. A Sentiment Analysis tab was added to the AI Bible Study Companion, classifying all 25 verses of Jude and visualising the distribution as a bar chart. The results reflect Jude's structure accurately, the doxology passages score positive while the judgment warnings score negative, showing how attention-based models capture tone without any task-specific training on biblical text.